Claude Code Hook 模块详解教程

目录

1. 什么是 Hook?
2. Hook 完整架构
3. 七种事件类型详解
4. hooks.json 配置格式
5. 编写第一个 Hook
6. Profile 控制系统
7. ECC 内置 Hook 详解
8. 实战食谱（Recipe）
9. run-with-flags.js 执行器
10. 跨平台注意事项
11. 调试与故障排除

---

1. 什么是 Hook?

Hook 是 Claude Code 的**事件驱动自动化机制**——在工具执行的关键节点自动触发脚本，完成质量门禁、错误拦截、格式化等任务。

核心价值

场景	没有 Hook	有 Hook
提交代码	可能包含 `console.log`	自动检测并阻止
修改 ESLint 配置	可能误改配置	自动拦截保护
运行 `npm run dev`	终端失控	强制在 tmux 中运行
编辑 .ts 文件	类型错误游离	立刻运行 `tsc` 检查
提交前	可能跳过 pre-commit	拦截 `--no-verify` 绕过

工作原理

```
用户请求 → Claude 选工具 → PreToolUse Hook → 工具执行 → PostToolUse Hook
                                                  ↓
                                              工具失败?
                                              → PostToolUseFailure Hook
                                                  ↓
                                            Claude 响应后?
                                              → Stop Hook (每轮响应后)
                                                  ↓
                                            上下文压缩前?
                                              → PreCompact Hook
                                                  ↓
```

```
                                                会话开始/结束?
                                                  → SessionStart / SessionEnd Hook
```

---

2. Hook 完整架构

ECC 的 Hook 系统由三部分组成:

```
┌──────────────────────────────────────────┐
│  hooks/hooks.json                        │
│  定义: 什么事件 + 什么工具 + 执行什么命令     │
└──────────────────────────────────────────┘
                │
                │ 读取配置
                ▼
┌──────────────────────────────────────────┐
│  scripts/hooks/run-with-flags.js          │
│  执行器: Profile 过滤 + 传递给具体脚本       │
└──────────────────────────────────────────┘
                │
                │ 检查 ECC_HOOK_PROFILE / ECC_DISABLED_HOOKS
                ▼
┌──────────────────────────────────────────┐
│  scripts/hooks/*.js                      │
│  具体业务逻辑脚本                           │
│  (PreToolUse / PostToolUse / Stop / etc.)  │
└──────────────────────────────────────────┘
```

文件结构

```
hooks/
├── hooks.json        ← 主配置文件 (声明式)
└── README.md         ← 文档

scripts/
├── hooks/            ← 所有 Hook 脚本
│   ├── run-with-flags.js        ← Profile 执行器 (核心)
│   ├── run-with-flags-shell.sh  ← Shell 版执行器
│   ├── check-hook-enabled.js    ← 启用状态检查
│   ├── pre-bash-tmux-reminder.js      ← PreToolUse
│   ├── pre-bash-git-push-reminder.js  ← PreToolUse
│   ├── pre-bash-commit-quality.js    ← PreToolUse (blocking)
│   ├── pre-bash-dev-server-block.js   ← PreToolUse (blocking)
│   ├── doc-file-warning.js            ← PreToolUse
│   ├── suggest-compact.js             ← PreToolUse
│   ├── config-protection.js           ← PreToolUse (blocking)
│   ├── post-edit-format.js            ← PostToolUse
│   ├── post-edit-typecheck.js         ← PostToolUse
│   ├── post-bash-pr-created.js        ← PostToolUse
│   ├── quality-gate.js                ← PostToolUse
│   ├── session-start.js               ← SessionStart
```

```
│   ├── session-end.js                ← SessionEnd
│   └── cost-tracker.js               ← Stop
└── lib/
    ├── hook-flags.js     ← Profile 控制系统
    ├── package-manager.js ← 包管理器检测
    └── ...
```

---

3. 七种事件类型详解

3.1 PreToolUse

触发时机： 工具执行之前

核心能力： 可以**阻止**工具执行（exit code 2）

```
// 输入数据结构
{
  tool_name: "Bash",
  tool_input: {
    command: "npm run dev"  // 即将执行的命令
  }
}
```

Exit Code 含义：

Exit Code	行为
`0`	放行，继续执行工具
`2`	**阻止**工具执行（阻塞）
其他	工具继续执行，但错误被记录

典型用途： 质量门禁、安全检查、配置保护

3.2 PostToolUse

触发时机： 工具成功执行之后

核心能力： 可访问 `tool_output`，但**无法阻止**工具执行

```
// 输入数据结构（比 PreToolUse 多 tool_output）
{
  tool_name: "Bash",
  tool_input: { command: "npm test" },
  tool_output: {
    output: "PASS 5 tests"  // 工具的输出结果
```

```
    }
  }
```

典型用途： 自动格式化、自动类型检查、日志记录

3.3 PostToolUseFailure

触发时机： 工具执行失败后

数据结构： 与 `PostToolUse` 相同，`tool_output` 包含错误信息

典型用途： 失败分析、告警通知、错误日志

3.4 Stop

触发时机： 每次 Claude 响应之后（每轮对话结束）

特点： 无 `tool_input`/`tool_output`，只有 `transcript_path`

```
{
  transcript_path: "/path/to/.claude/transcripts/xxx.json"
}
```

典型用途： 会话总结、成本追踪、模式提取

3.5 PreCompact

触发时机： 上下文压缩之前

目的： 保存必要的会话状态，避免压缩时丢失关键信息

典型用途： 状态持久化、上下文摘要保存

3.6 SessionStart

触发时机： 会话开始时（每次启动 Claude Code）

典型用途： 加载上次的会话上下文、检测包管理器、初始化状态

3.7 SessionEnd

触发时机： 会话结束时

典型用途： 会话状态持久化、总结提取、清理工作

---

4. hooks.json 配置格式

完整结构

```
{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash|Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "node scripts/hooks/my-hook.js",
            "async": false,
            "timeout": 30
          }
        ],
        "description": "我的自定义 Hook 描述"
      }
    ]
  }
}
```

matcher 语法

```
"matcher": "Bash"                 // 单一工具
"matcher": "Edit|Write"           // 多个工具（用 | 分隔）
"matcher": "Write|Edit|MultiEdit" // 多个写操作工具
"matcher": "*"                    // 所有工具
```

所有支持的工具名

类别	工具名
文件操作	`Read`、`Write`、`Edit`、`MultiEdit`、`Bash`、`Glob`、`Grep`
任务管理	`Task`、`TaskOutput`、`TaskCreate`、`TaskUpdate`、`TaskList`
其他	`WebFetch`、`WebSearch`、`NotebookEdit` 等

async 与 timeout

```
{
  "type": "command",
  "command": "node my-slow-hook.js",
  "async": true,     // 异步执行，不阻塞
```

```
  "timeout": 30       // 超时秒数（默认 30s）
}
```

组合	效果
`async: false`（默认）	同步阻塞，等 Hook 完成后才继续
`async: true`	后台异步执行，不阻塞，timeout 控制最大等待

---

5. 编写第一个 Hook

最小模板（Node.js）

```
#!/usr/bin/env node
/**
 * PreToolUse Hook 示例：阻止包含密码的 git commit
 */

'use strict';

// 读取 stdin（Hook 输入是 JSON）
let raw = '';
process.stdin.setEncoding('utf8');
process.stdin.on('data', chunk => { raw += chunk; });
process.stdin.on('end', () => {
  try {
    const input = JSON.parse(raw);
    const cmd = input.tool_input?.command || '';

    // === 业务逻辑 ===
    const dangerous = /password|secret|api_key|token\s*=/i.test(cmd);

    if (dangerous && cmd.includes('git commit')) {
      // 阻止执行
      console.error('[Hook] 阻止提交：检测到疑似密码/密钥信息');
      process.exit(2);  // exit 2 = 阻止
    }

    // === 放行：原样输出 ===
    console.log(raw);
  } catch (err) {
    // 出错也要放行，不能破坏用户流程
    console.log(raw);
  }
});
```

现代写法（推荐：导出 run() 函数）

```
#!/usr/bin/env node
'use strict';

const MAX_STDIN = 1024 * 1024;

function run(rawInput, options = {}) {
  try {
    const input = JSON.parse(rawInput);
    const toolName = input.tool_name;
    const toolInput = input.tool_input || {};

    // === 业务逻辑 ===
    // 例如: 阻止在非 tmux 环境下运行 dev server
    if (toolName === 'Bash') {
      const cmd = toolInput.command || '';
      const devServerPattern = /npm run dev|yarn dev|pnpm dev|npm start/i;
      if (devServerPattern.test(cmd)) {
        // 检查是否在 tmux 中
        if (!process.env.TMUX) {
          console.error('[Hook] 阻止: 请在 tmux 中运行开发服务器');
          return { exitCode: 2, stderr: 'Blocked', output: rawInput };
        }
      }
    }

    return rawInput; // 放行
  } catch (err) {
    return rawInput; // 出错放行
  }
}

// stdin 入口 (向后兼容)
if (require.main === module) {
  let data = '';
  process.stdin.on('data', c => { data += c; });
  process.stdin.on('end', () => {
    const result = run(data);
    if (result.output) process.stdout.write(result.output);
    process.exit(result.exitCode || 0);
  });
}

module.exports = { run };
```

注册到 hooks.json

```
{
  "hooks": {
    "PreToolUse": [
      {
```

```
          "matcher": "Bash",
          "hooks": [
            {
              "type": "command",
              "command": "node /absolute/path/to/my-hook.js"
            }
          ],
          "description": "阻止包含密码信息的 git commit"
        }
      ]
    }
  }
```

调试技巧：内联 Hook（无需创建文件）

```
{
  "matcher": "Edit",
  "hooks": [{
    "type": "command",
    "command": "node -e \"let
d='';process.stdin.on('data',c=>d+=c);process.stdin.on('end',()=>{const
i=JSON.parse(d);const
ns=i.tool_input?.new_string||'';if(/TODO|FIXME|HACK/.test(ns))
{console.error('[Hook] 检测到 TODO/FIXME')}console.log(d)})\"  }]
}
```

---

6. Profile 控制系统

ECC 通过两个环境变量控制 Hook 的启用行为，**无需修改 hooks.json。**

6.1 ECC_HOOK_PROFILE

```
export ECC_HOOK_PROFILE=standard  # 默认值
```

Profile	描述	包含的 Hook
`minimal`	仅保留关键安全 Hook	SessionStart/End、PostEditFormat 等
`standard`	平衡质量与便利（默认）	minimal + tmux 提醒、commit 质量检查等
`strict`	最高安全级别	standard + 所有额外提醒和严格门禁

6.2 ECC_DISABLED_HOOKS

```
# 禁用多个 Hook（逗号分隔）
export ECC_DISABLED_HOOKS="pre:bash:tmux-reminder,post:edit:typecheck"
```

```
# 禁用所有 TypeScript 检查
export ECC_DISABLED_HOOKS="post:edit:typecheck"
```

6.3 run-with-flags.js 的作用

`hooks.json` 中的 Hook 通常不直接指向业务脚本，而是通过 `run-with-flags.js` 包装：

```
{
  "type": "command",
  "command": "node \"${CLAUDE_PLUGIN_ROOT}/scripts/hooks/run-with-flags.js\"
\"pre:bash:tmux-reminder\" \"scripts/hooks/pre-bash-tmux-reminder.js\" \"strict\""
}
```

参数说明：

```
run-with-flags.js <hookId> <scriptPath> <profilesCsv>
                    ↑         ↑               ↑
              唯一标识符   实际业务脚本    在哪些 Profile 下启用
```

执行流程：

```
hooks.json 命令
  ↓
run-with-flags.js 读取 ECC_HOOK_PROFILE
  ↓
检查 hookId 是否在 ECC_DISABLED_HOOKS 中
  ↓
检查当前 profile 是否在允许列表中
  ↓
三者都通过 → 执行真实业务脚本
任一不通过 → 直接放行（不执行）
```

---

7. ECC 内置 Hook 详解

7.1 PreToolUse Hooks

dev-server-block（阻塞）

```
{
  "matcher": "Bash",
  "hooks": [{ "type": "command", "command": "node pre-bash-dev-server-block.js" }]
}
```

功能： 阻止 `npm run dev`、`yarn dev` 等开发服务器命令**在 tmux/screen 外运行。**

原理： 检测命令是否匹配 dev server 模式 → 检查 `$TMUX`/`$STY` 环境变量 → 无则 exit 2。

为什么需要： 开发服务器运行时 Ctrl+C 会终止进程；在 tmux 中运行可保持后台。

---

pre-bash-commit-quality（阻塞）

```
{
  "matcher": "Bash",
  "hooks": [{ "type": "command", "command": "node pre-bash-commit-quality.js" }]
}
```

功能： 在 `git commit` 之前运行三道质量门禁：

检查项	行为
暂存区文件有 console.log/debugger	阻止（exit 2）
检测到疑似密钥（password=、api_key 等）	阻止
暂存文件有 ESLint/Ruff 错误	阻止（exit 2）
提交信息格式不规范（conventional commit）	警告（exit 0）

---

pre-bash-tmux-reminder（警告）

条件： Profile = `strict`

功能： 检测到 `npm test`、`cargo build`、`docker run` 等长时间命令时，提醒使用 tmux。

---

pre-bash-git-push-reminder（警告）

条件： Profile = `strict`

功能： `git push` 前提醒先审查 diff。

---

doc-file-warning（警告）

条件： Profile = `standard|strict`

功能： 创建非标准文档文件时警告（如 `NOTES.md`、`TODO.md`），允许 README/CLAUDE/CONTRIBUTING/CHANGELOG/LICENSE 等标准文件。

---

suggest-compact（警告）

条件： Profile = `standard|strict`

功能： 每约 50 次工具调用后，建议用户手动执行 `/compact`。

---

config-protection（阻塞）

功能： 阻止修改 ESLint/Prettier/Biome/Ruff 等配置文件，防止格式化规则被意外更改。

7.2 PostToolUse Hooks

post-edit-format（自动格式化）

触发： `Edit` 工具

功能： 编辑 JS/TS 文件后，自动用 Biome（首选）或 Prettier 格式化：

```
// 核心逻辑（伪代码）
if (toolOutput.edit?.length > 0) {
  const formatter = detectFormatter(); // biome > prettier > 无
  if (formatter === 'biome') {
    execSync(`npx @biomejs/biome format --write ${filePath}`);
  } else if (formatter === 'prettier') {
    execSync(`npx prettier --write ${filePath}`);
  }
}
```

---

post-edit-typecheck（类型检查）

触发： `Edit` 工具修改 `.ts`/`.tsx` 文件

功能： 自动运行 `tsc --noEmit` 检查类型错误，**不会阻止**，只警告。

---

post-edit-console-warn（警告）

触发： `Edit` 工具

功能： 检测编辑后的文件中新增了 `console.log` 语句，提醒清理。

---

post-bash-pr-created（日志）

触发： `Bash` 执行了 `gh pr create`

功能： 提取 PR URL，记录到会话日志。

7.3 Lifecycle Hooks

session-start

触发： 会话启动

功能：

- 读取上次的会话摘要（如果有）
- 检测项目包管理器（npm/pnpm/yarn/bun）
- 打印欢迎信息

pre-compact

触发： 上下文压缩前

功能： 保存当前会话状态到 `$CLAUDE_PROJECT_DIR/.claude/sessions/`，确保压缩后关键信息不丢失。

cost-tracker（Stop Hook）

触发： 每轮 Claude 响应后

功能： 记录 token 使用量和估算成本到会话元数据。

---

8. 实战食谱（Recipe）

Recipe 1：阻止超过 800 行的文件创建

```
{
  "matcher": "Write",
  "hooks": [{
    "type": "command",
    "command": "node -e \"let
d='';process.stdin.on('data',c=>d+=c);process.stdin.on('end',()=>{const
i=JSON.parse(d);const c=i.tool_input?.content||'';const
lines=c.split('\\n').length;if(lines>800){console.error('[Hook] 阻止: 文件超过800行
('+lines+'行)');process.exit(2)}console.log(d)})\"  }],
  "description": "阻止创建超大型文件"
}
```

Recipe 2：强制新 .ts 文件必须有测试文件

```
{
  "matcher": "Write",
  "hooks": [{
    "type": "command",
    "command": "node -e \"const fs=require('fs');let
d='';process.stdin.on('data',c=>d+=c);process.stdin.on('end',()=>{const
i=JSON.parse(d);const p=i.tool_input?.file_path||'';if(/src\\/.*\\.
```

```
(ts|tsx)$/.test(p)&&!/\\.test\\.|\\.spec\\./.test(p)){const tp=p.replace(/\\.
(ts|tsx)$/,'.test.$1');if(!fs.existsSync(tp)){console.error('[Hook] 新增源文件但无
测试: '+p+', 建议使用 /tdd 先写测试')}}console.log(d)})\""  }],
  "description": "提醒创建测试文件"
}
```

Recipe 3：自动用 ruff 格式化 Python 文件

```
{
  "matcher": "Edit",
  "hooks": [{
    "type": "command",
    "command": "node -e \"const{execFileSync}=require('child_process');let
d='';process.stdin.on('data',c=>d+=c);process.stdin.on('end',()=>{const
i=JSON.parse(d);const p=i.tool_input?.file_path||'';if(/\\.py$/.test(p))
{try{execFileSync('ruff',['format',p],{stdio:'pipe'})}catch(e)
{}}console.log(d)})\""  }],
  "description": "Python 编辑后自动 ruff format"
}
```

Recipe 4：git commit 时自动 conventional commit 校验

```
{
  "matcher": "Bash",
  "hooks": [{
    "type": "command",
    "command": "node -e \"let
d='';process.stdin.on('data',c=>d+=c);process.stdin.on('end',()=>{const
i=JSON.parse(d);const cmd=i.tool_input?.command||'';if(/git commit/.test(cmd))
{const msg=cmd.match(/--message=['\\\"](.*?)['\\\"]/)?.[1];const
pattern=/^(feat|fix|docs|style|refactor|test|chore)(\\(.+\\))?:\\s+.
{1,50}/;if(msg&&!pattern.test(msg)){console.error('[Hook] 提交信息格式不规
范');console.error('[Hook] 使用 conventional commit: type(scope):
description');console.error('[Hook] 例如: feat(auth): add login
endpoint');process.exit(2)}}console.log(d)})\""  }],
  "description": "git commit 时强制 conventional commit 格式"
}
```

Recipe 5：PostToolUse 检测测试是否通过

```
{
  "matcher": "Bash",
  "hooks": [{
    "type": "command",
```

```
    "command": "node -e \"let
d='';process.stdin.on('data',c=>d+=c);process.stdin.on('end',()=>{const
i=JSON.parse(d);const cmd=i.tool_input?.command||'';const
out=i.tool_output?.output||'';if(/npm test|npx
vitest|jest/.test(cmd)&&out.includes('FAIL')){console.error('[Hook] 测试失败，请修
复后再提交');process.exit(2)}console.log(d)})\""  }],
  "description": "npm test 失败时阻止继续"
}
```

Recipe 6：启动会话时加载上次摘要

```
{
  "matcher": "SessionStart",
  "hooks": [{
    "type": "command",
    "command": "node -e \"const fs=require('fs');const path=require('path');const
summary=path.join(process.env.HOME,'.claude','sessions','last-
summary.txt');if(fs.existsSync(summary)){console.error('[Session] 上次会话摘
要:');console.error(fs.readFileSync(summary,'utf8').trim())}console.log('{}')\"",
    "async": true,
    "timeout": 5
  }],
  "description": "SessionStart: 显示上次会话摘要"
}
```

---

9. run-with-flags.js 执行器

这是 ECC Hook 系统的核心，理解它有助于自定义复杂 Hook。

核心逻辑

```
// run-with-flags.js 伪代码
function main(hookId, scriptPath, allowedProfilesCsv) {
  // Step 1: 检查是否被禁用
  if (isHookDisabled(hookId)) {
    return passThrough(); // 直接放行
  }

  // Step 2: 检查 profile
  const currentProfile = process.env.ECC_HOOK_PROFILE || 'standard';
  if (!currentProfileAllows(allowedProfilesCsv, currentProfile)) {
    return passThrough(); // 直接放行
  }

  // Step 3: 读取 stdin 并传递给真实脚本
  const input = readStdin();
  const result = spawnSync('node', [scriptPath], {
```

```
    input,
    stdio: ['pipe', 'pipe', 'pipe'],
    timeout: 30_000
  });

  // Step 4: 处理返回值
  return result;
}
```

快速禁用任何 Hook（无需改 hooks.json）

```
# 通过环境变量禁用
export ECC_DISABLED_HOOKS="pre:bash:tmux-reminder,pre:bash:git-push-reminder"
```

这比修改 `hooks.json` 更安全——更新 ECC 时不会被覆盖。

---

10. 跨平台注意事项

Node.js 是首选

ECC 所有 Hook 都用 Node.js 实现，保证 Windows/macOS/Linux 行为一致。

Shell 脚本的注意事项

少量 Hook 使用 shell 脚本（`run-with-flags-shell.sh`），这些在 Windows 上可能失效。**建议 Hook 脚本都使用 Node.js。**

Windows 路径处理

```
// 错误：Linux/macOS 风格路径
const filePath = '/path/to/file.txt';

// 正确：使用 path.join
const path = require('path');
const filePath = path.join(process.cwd(), 'file.txt');
```

spawnSync 的跨平台调用

```
const { spawnSync } = require('child_process');

// Linux/macOS
spawnSync('npx', ['eslint'], { stdio: 'pipe' });

// Windows（npx 在 Windows 上行为可能不同）
// 更好的做法：使用 Node.js 等效工具
```

TMUX 检测的 Windows 等效

```
// Linux/macOS
if (process.env.TMUX || process.env.STY) { ... }

// Windows 等效：检测 Windows Terminal / PowerShell 远程会话
if (process.env.TERM_PROGRAM === 'vscode' ||
    process.env.WSENV) { /* 可选增强 */ }
```

---

11. 调试与故障排除

11.1 查看哪些 Hook 在运行

```
# 启用调试输出（Hook 会打印启动信息）
export DEBUG=hooks:*
claude
```

11.2 验证 Hook 是否生效

在 hooks.json 中加入一个无害的测试 Hook：

```
{
  "matcher": "Bash",
  "hooks": [{
    "type": "command",
    "command": "node -e
\"console.log(JSON.stringify(JSON.parse(require('fs').readFileSync('/dev/stdin','u
tf8'))));console.error('[DEBUG] Bash hook triggered!')\""
  }],
  "description": "调试：Bash 工具触发时打印信息"
}
```

11.3 常见错误

症状	原因	解决
Hook 完全没有触发	`hooks.json` 未被加载	确认文件在 `~/.claude/` 或项目 `.claude/` 目录
stdin 数据为空	Node.js 脚本未正确 `pipe` stdin	检查 `process.stdin.setEncoding('utf8')`
exit 2 但工具仍执行	只能是 PreToolUse，PostToolUse 的 exit 2 无效	确认事件类型

症状	原因	解决
Windows 上 Hook 失败	shell 脚本不兼容	改用 Node.js 脚本
大量 Hook 拖慢速度	同步串行执行	对非必需的 Hook 改用 `async: true`

11.4 性能优化

- **对不阻塞的 Hook 使用 `async: true`**：格式化、检查等 Hook 都异步执行
- **对不重要的 Hook 设置短 timeout**：`timeout: 5` 防止卡死
- **PostToolUse 优于 PreToolUse**：格式化尽量在 PostToolUse 中做，不阻塞编辑
- **避免在 PreToolUse 中做网络请求**：太慢会影响响应速度

11.5 快速禁用全部 Hook

```
# 完全禁用（测试用）
export ECC_DISABLED_HOOKS="pre:bash:tmux-reminder,pre:bash:git-push-reminder,pre:bash:commit-quality,pre:edit-write:suggest-compact,pre:write:doc-file-warning,post:edit:format,post:edit:typecheck"
```

附录：工具输入字段速查

工具	关键 tool_input 字段
`Bash`	`command`（完整命令字符串）
`Write`	`file_path`（目标路径），`content`（文件内容）
`Edit`	`file_path`, `old_string`, `new_string`
`Read`	`file_path`
`Glob`	`pattern`
`Grep`	`path`, `pattern`
`Task`	`prompt`, `subagent_type`, `description`

附录：Exit Code 含义速查

Exit Code	PreToolUse	PostToolUse	Stop	SessionStart/End
`0`	放行	—	—	—
`2`	**阻止执行**	无效（被忽略）	无效	无效
其他	记录错误，放行	记录错误，继续	无效	无效